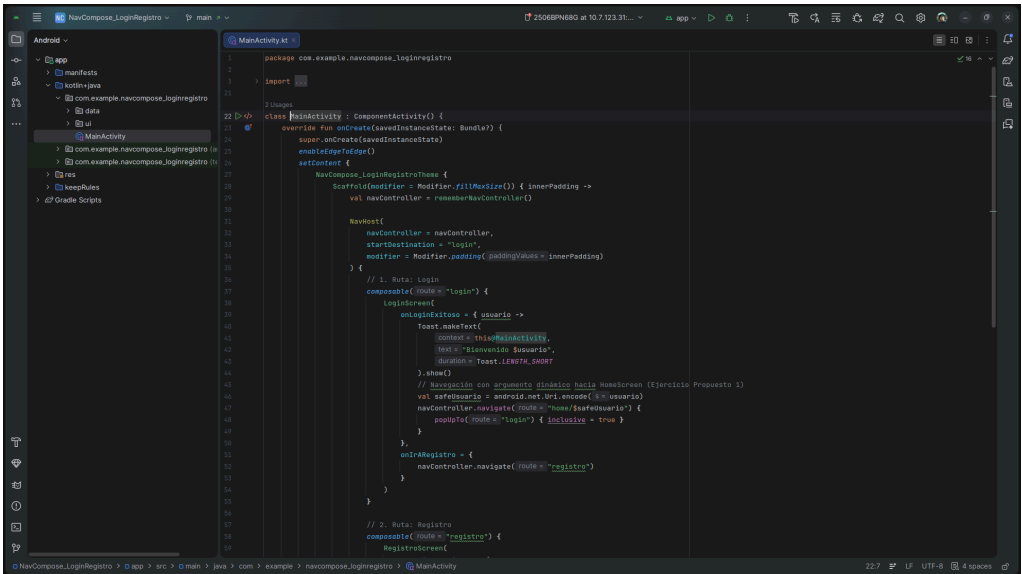


	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	Introducción al desarrollo de nuevas plataformas				
TÍTULO DE LA PRÁCTICA:	Navegación entre pantallas: inicio de sesión y registro con Jetpack Compose				
NÚMERO DE LA PRÁCTICA:	3	AÑO LECTIVO:	2026	NRO. SEMESTRE:	B
FECHA DE PRESENTACIÓN:	15/09/2026	HORA DE PRESENTACIÓN:	11:59:00		
INTEGRANTE(s): - Jorge Luis Mamani Huarsaya - Yordano Hernan Boza Portilla				NOTA (0 - 20):	Nota colocada por el docente
DOCENTE: Roxana Evelyn Limache Calatayud					

RESULTADOS Y PRUEBAS
Ejercicio 1: Creación de HomeScreen y paso de parámetros Para este ejercicio, necesitábamos implementar una tercera pantalla (HomeScreen) en el grafo de navegación que fuera capaz de recibir dinámicamente el nombre del usuario y mostrarlo como mensaje de bienvenida tras un inicio de sesión exitoso. Primero, agregamos la ruta "home/{usuario}" dentro de nuestro NavHost en el archivo MainActivity.kt. Para recibir el parámetro de forma segura, declaramos la variable a través de <code>navArgument("usuario")</code> especificando que sería del tipo <code>StringType</code>. Una vez extraído el parámetro del <code>backStackEntry</code>, se lo inyectamos directamente al composable HomeScreen. Además de esto, observamos que para mantener una navegación coherente y evitar que el usuario vuelva a la pantalla de "Login" presionando el botón físico de atrás en su dispositivo, debíamos destruir esa ruta de la pila. Para ello, utilizamos <code>popUpTo("login") { inclusive = true }</code>.  Figura 1: Código del NavHost en MainActivity

Posteriormente, desarrollamos la vista `HomeScreen.kt`, la cual recibe la cadena del usuario y la concatena en un texto utilizando `MaterialTheme`.

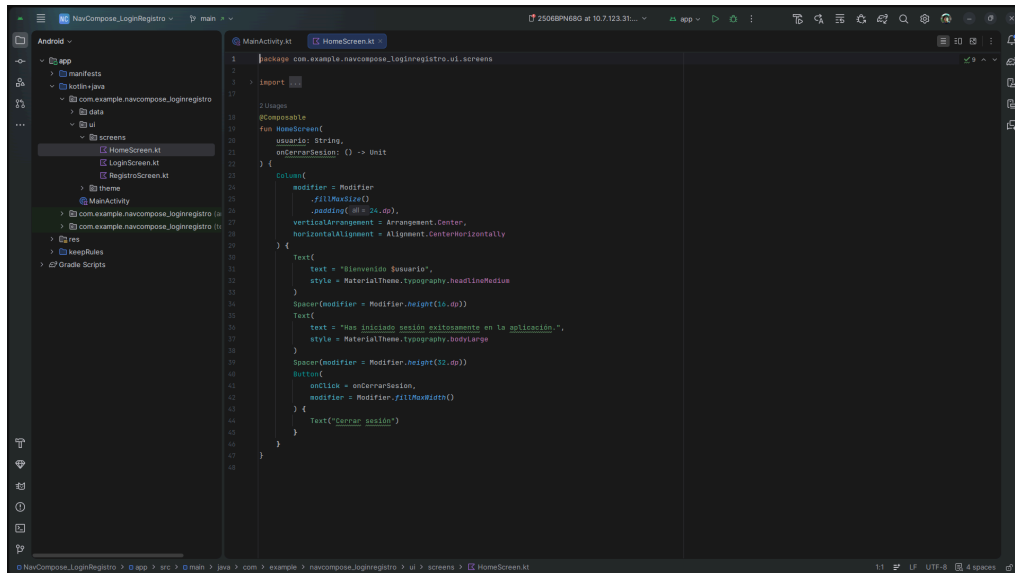


Figura 2: Código del screen HomeScreen

Realizamos un ejemplo donde el usuario realiza un registro de cuenta y luego puede realizar el inicio de sesión. Podremos observar que los datos son pasados con éxito.

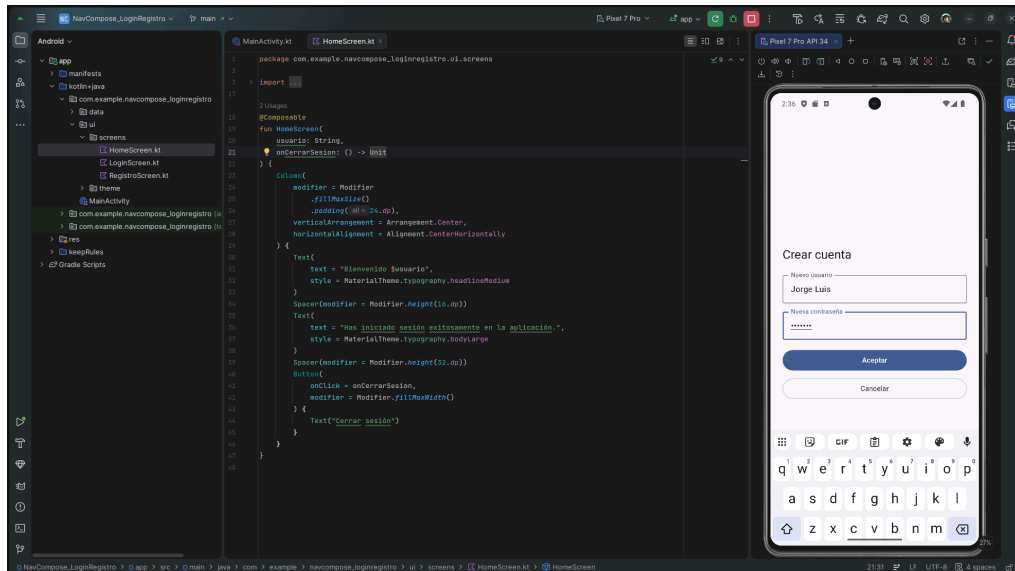


Figura 3: Pantalla de Registro de cuenta

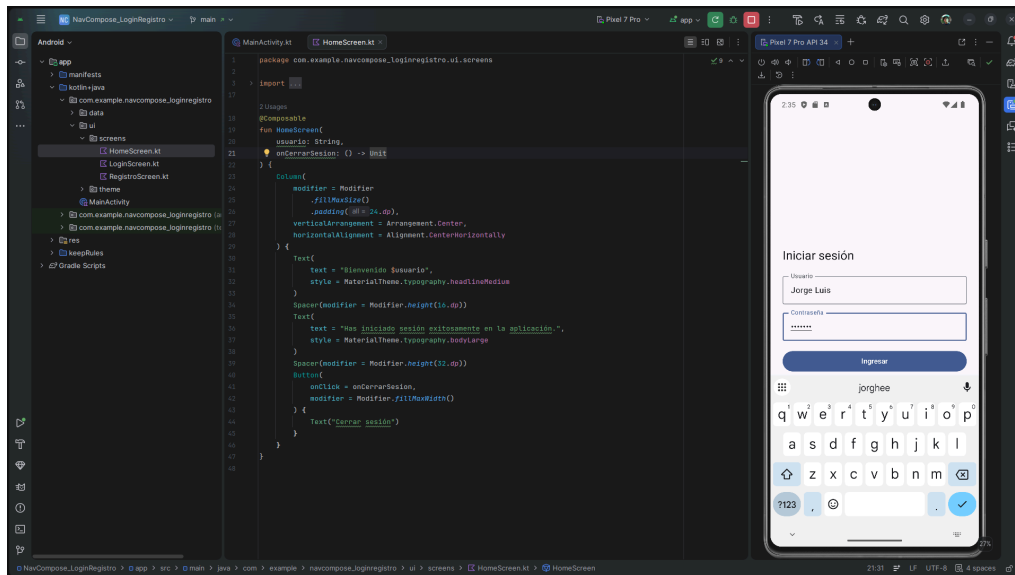


Figura 4: Pantalla de Inicio de sesión

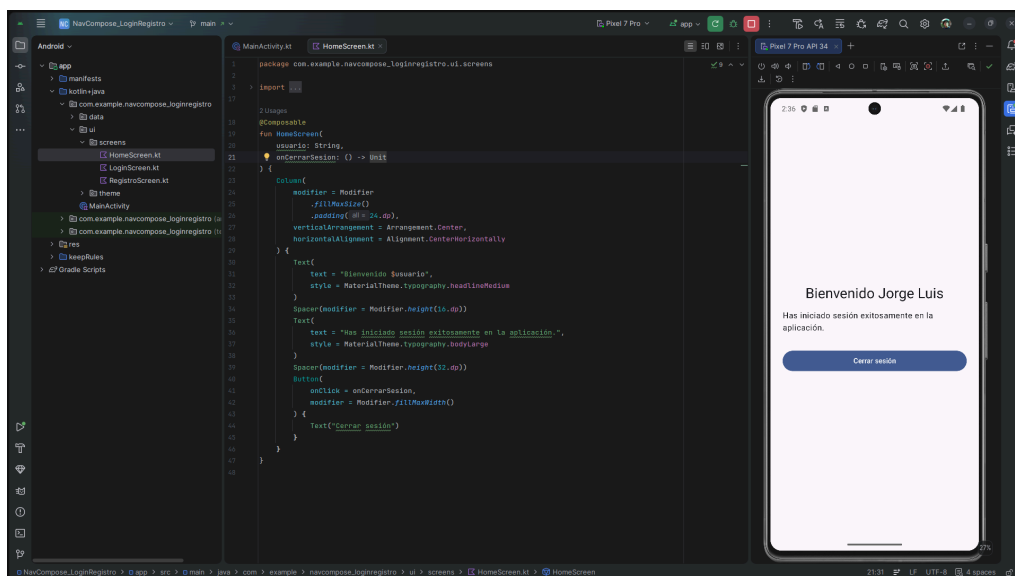


Figura 5: Pantalla de HomeScreen con los datos dinámicos

Ejercicio 2: Persistencia de datos en Registro

La implementación de la persistencia de datos en registro consistirá en los siguientes pasos: al presionar “Aceptar” en la pantalla de registro, guardáramos las credenciales del usuario en un archivo plano de texto (cuentas.txt) dentro del almacenamiento interno de la app utilizando MODE_APPEND.

Para mantener un proyecto ordenado y seguir buenas prácticas de la arquitectura en Android, decidimos crear un gestor de persistencia que actuara como única fuente de la verdad (Single Source of Truth). Implementamos el objeto AccountManager en un nuevo paquete data/.

Dentro de AccountManager.kt, creamos la función guardarCuenta(), la cual recibe el Contexto para acceder al almacenamiento y escribe el binomio usuario-contraseña concatenado con un delimitador.

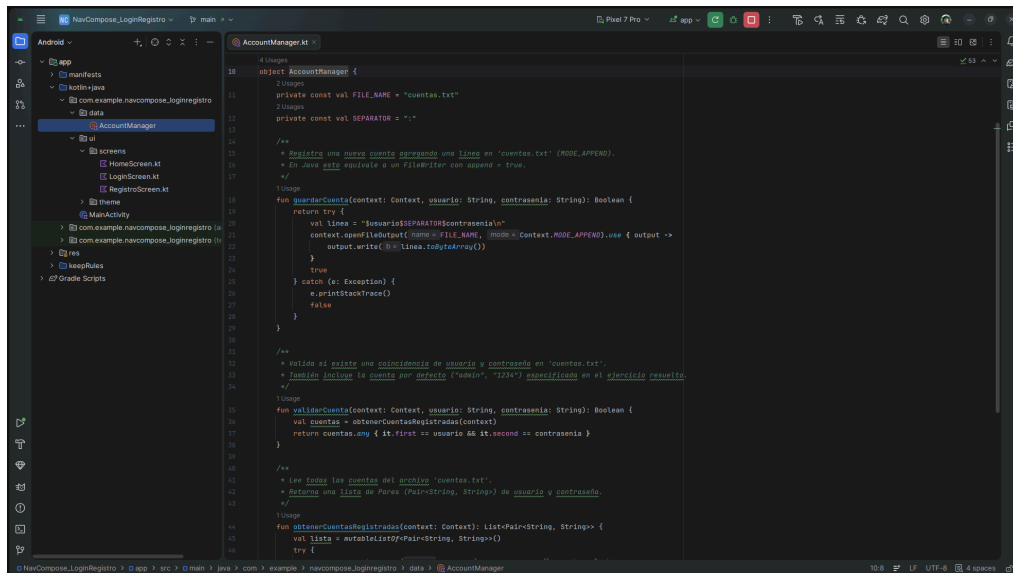


Figura 6: Función guardarCuenta con MODE_APPEND

Luego, en RegistroScreen.kt, recuperamos el contexto de Compose mediante `LocalContext.current` y lo utilizamos para llamar a nuestra función de guardado antes de navegar hacia atrás (`navController.popBackStack()`). Comprobamos que esto soluciona el requerimiento sin acoplar la lógica de escritura directamente en la vista.

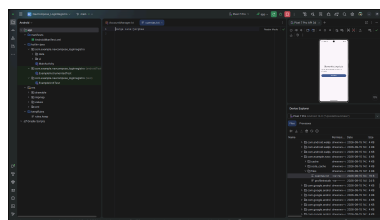


Figura 7: Toast confirmando registro exitoso

Ejercicio 3: Validación de Login mediante archivo plano

El tercer ejercicio consistía en modificar la validación estática de la pantalla de inicio de sesión para que el acceso se determinara contrastando las credenciales únicamente contra el archivo `cuentas.txt` y ya no mediante una lista en memoria.

Para ello, implementamos la función `obtenerCuentasRegistradas()` en nuestro `AccountManager`. Esta función abre el archivo utilizando `context.openFileInput(FILE_NAME)`, lo lee línea por línea con un `BufferedReader` y divide cada línea mediante el delimitador configurado para reconstruir una lista de objetos tipo `Pair<String, String>`.

Posteriormente, en la función de control principal `validarCuenta()`, verificamos si las credenciales coinciden iterando sobre las tuplas obtenidas.

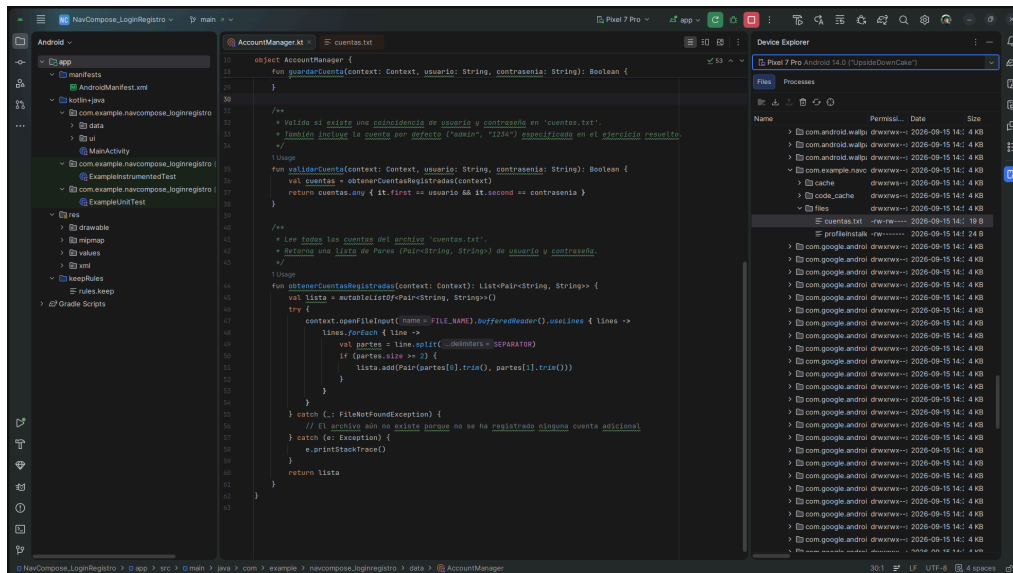


Figura 8: Función validarCuenta contrastando credenciales

En la capa de presentación (LoginScreen.kt), dependiendo de la respuesta booleana devuelta por el manager, enrutamos al usuario hacia HomeScreen o pintamos el texto de error “Cuenta no encontrada o credenciales incorrectas”.

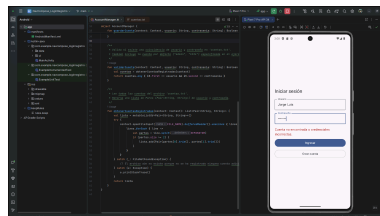


Figura 9: Transición errónea tras iniciar sesión validando con archivo

Ejercicio 4: Reto Opcional (Validación de campos vacíos)

Finalmente, asumimos el reto opcional que requería agregar una validación de campos incompletos para los formularios de ambas pantallas antes de procesar cualquier información o intentar interactuar con el sistema de archivos.

Lo resolvimos agregando una validación reactiva empleando los estados gestionados por Jetpack Compose. Implementamos el chequeo `if (usuario.isBlank() || password.isBlank())` tanto en el evento `onClick` del botón “Ingresar” (LoginScreen) como en el botón “Aceptar” (RegistroScreen).

Si la evaluación resulta verdadera, interrumpimos el flujo con un `return@Button` y asignamos el mensaje “Por favor, llene todos los campos.” a la variable mutable de estado `mensajeError`, lo que hace que Compose re-dibuje la interfaz instantáneamente mostrando la advertencia de color rojo al usuario sin hacer crashear la aplicación.

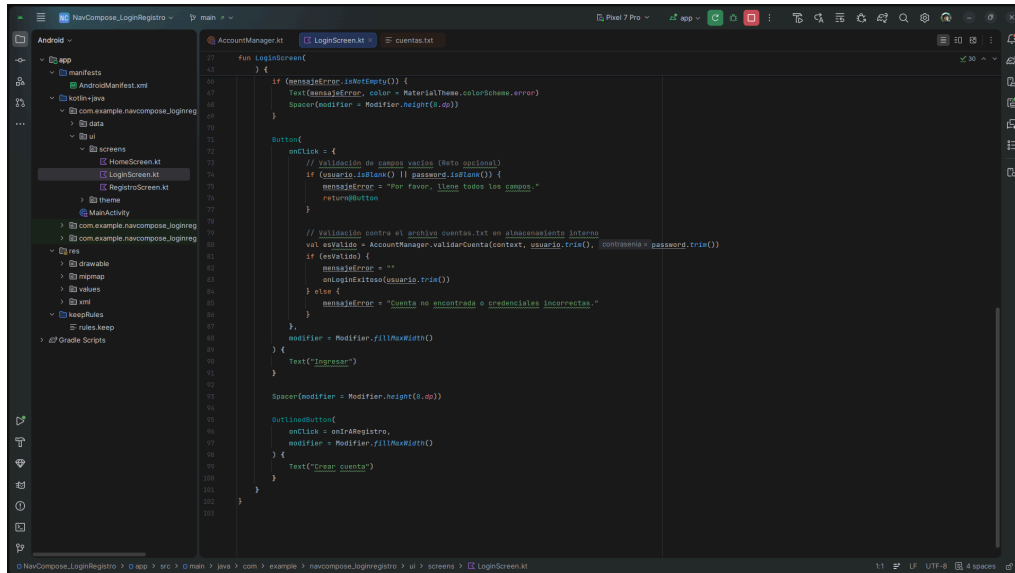


Figura 10: Retorno anticipado ante campos vacíos

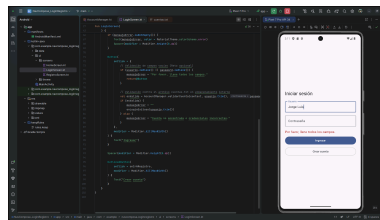


Figura 11: Mensaje de error reactivo en interfaz

CUESTIONARIO

1. ¿Qué ventajas tiene usar Navigation Compose (una sola Activity con varias pantallas) frente al enfoque tradicional de múltiples Activities comunicadas por Intent?

La adopción de una arquitectura de una sola Activity (Single-Activity Architecture) impulsada por Navigation Compose nos ofreció varias ventajas importantes frente al enfoque clásico:

1. Menor sobrecarga del sistema (Overhead): Crear y destruir Activities es un proceso pesado para el sistema operativo Android. Al usar Composables, simplemente estamos re-dibujando funciones en memoria sobre el mismo lienzo, lo que hace que la transición entre pantallas sea significativamente más rápida y fluida.
2. Gestión centralizada del estado: En lugar de tener que serializar objetos pesados mediante Intents o Bundles para pasarlos entre Activities, en Compose gran parte de la jerarquía comparte el mismo contexto.
3. Flujo declarativo: Al definir todo en un solo NavHost, tuvimos visibilidad completa de todo el grafo de navegación del aplicativo en un único archivo (MainActivity.kt), lo que nos facilita escalar y mantener el código.

2. Además de los argumentos de navegación, ¿qué otro mecanismo podrías usar para compartir datos entre pantallas en una app Compose (por ejemplo, un ViewModel compartido)? Explica brevemente cómo funcionaría.

Aunque los argumentos de navegación (como usamos en `navArgument("usuario")`) son útiles para datos primarios o simples como Strings y Enteros, no son el mecanismo recomendado para objetos complejos o dominios pesados. Para ello, utilizaríamos un ViewModel compartido.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 7

¿Cómo funcionaría? Podríamos instanciar un `ViewModel` a nivel superior (por ejemplo, pasándolo como parámetro en el nivel del `NavHost`) o utilizando un `NavGraph` anidado. De este modo:

1. El `ViewModel` alojaría las variables de estado (por ejemplo, `StateFlow` o `LiveData`) que representan la sesión del usuario.
2. La pantalla de `LoginScreen` inyectaría el `ViewModel` y, tras un inicio de sesión exitoso, actualizaría su estado.
3. Al navegar a `HomeScreen`, esta pantalla también inyectaría el mismo `ViewModel` (ya instanciado) y simplemente reaccionaría al estado previamente almacenado.

Esto nos liberaría de la responsabilidad de pasar los datos mediante la ruta del `NavController`, desacoplando completamente la lógica de estado de la lógica de navegación y permitiendo la supervivencia de los datos ante cambios de configuración (como girar la pantalla).

CONCLUSIONES

Durante el desarrollo del presente laboratorio, logramos consolidar nuestro entendimiento sobre la arquitectura de `Single-Activity` en Android al implementar con éxito la biblioteca `Navigation Compose`.

Comprobamos que declarar nuestras pantallas como simples funciones `@Composable` orquestadas por un `NavHost` simplifica drásticamente el flujo de trabajo en comparación con el uso de múltiples actividades e `Intents`. Adicionalmente, enfrentamos el reto de mantener la persistencia de datos mediante la escritura asíncrona en archivos del sistema (`Context.openFileOutput`). Esta implementación nos permitió comprender cómo el ecosistema de `Compose` interactúa con el contexto subyacente de Android, logrando un balance donde la vista (UI) y los datos (`AccountManager`) conviven separados bajo el principio de responsabilidad única.

REFERENCIAS

Bibliography